

TABLE OF CONTENT

S.NO.	TITLE	PAGE NO.
1.	Introduction	2
2.	Rationale	2
3.	Objectives	3
4.	Feasibility Study	3
5.	Methodology/ Planning of Work	4
6.	Software Required for the Project's Development	5
7.	Expected Outcomes	6

1. Introduction

- The NetScope project aims to build a real-time packet sniffing and network traffic analysis platform for monitoring and securing computer networks.
- It functions as a network monitoring and threat detection system for analyzing live network communication.
- It targets students, cybersecurity enthusiasts, administrators, and organizations requiring network visibility and traffic analysis.
- It captures and analyzes network packets to extract information such as IP addresses, protocols, ports, packet size, and traffic patterns.
- A key feature is the Real-Time Traffic Monitoring System which provides live insights into network activity and bandwidth usage.
- It includes a Smart Packet Filtering Engine that allows filtering traffic based on IP address, protocol, port, and packet type.
- Enhanced monitoring capabilities include suspicious activity detection, traffic statistics, and alert generation.
- The architecture is modular and microservices-based for scalability and efficiency.
- The project helps bridge the gap between theoretical networking concepts and practical cybersecurity implementation.

2. Rationale

The rationale behind developing this packet sniffer and network analyzer is to combine practical networking knowledge with real-world cybersecurity applications. The project not only serves as a learning platform for understanding protocols such as TCP, UDP, ICMP, DNS, and ARP, but also demonstrates how software engineering can be used to build scalable security-focused systems. Features such as real-time traffic visualization, intelligent filtering, threat detection, and packet logging make the system useful in academic environments, small organizations, and personal network management.

The project highlights the practical application of network programming, concurrent systems, and security analysis while addressing the growing need for accessible network observability tools.

3. Objectives

- To capture and analyze live network packets in real time from different network interfaces.
- To monitor network traffic and extract meaningful information such as IP addresses, protocols, ports, and packet statistics.
- To detect suspicious or malicious network activities including port scanning, flooding, and abnormal traffic behavior.
- To provide a user-friendly dashboard for real-time visualization, filtering, and logging of network traffic data.

4. Feasibility Study

Feasibility

The development of the packet sniffer and network traffic analyzer is technically and economically feasible due to the availability of open-source packet capture libraries, modern networking frameworks, and scalable backend technologies. The project can be implemented using commonly available systems and tools and modern frontend frameworks without requiring expensive hardware or infrastructure. Its modular architecture also allows easy testing, deployment, and future scalability.

Need

With the rapid growth of internet usage and connected devices, monitoring network traffic and identifying suspicious activities has become increasingly important. Many existing enterprise-level monitoring solutions are costly and difficult for students or small organizations to access. This project addresses the need for an affordable and practical system that helps users understand network behavior, analyze traffic patterns, and improve cybersecurity awareness through real-time packet analysis.

Significance

The project demonstrates the practical application of networking, cybersecurity, and software engineering concepts in a real-world environment. It provides valuable insights into how network communication occurs and how threats can be identified through packet-level analysis. The system can assist in network troubleshooting, traffic monitoring, security analysis, and educational learning, making it a useful tool for both academic and practical cybersecurity purposes.

5. Methodology/ Planning of work

Packet Capture and Traffic Analysis

The system captures live network packets from available network interfaces using packet capture libraries such as libpcap or Npcap. Captured packets are processed to extract important information including source and destination IP addresses, ports, protocols, packet size, and payload metadata. The traffic analysis module classifies packets based on protocol types such as TCP, UDP, ICMP, DNS, and ARP, enabling real-time monitoring of network communication and traffic behavior.

Threat Detection and Visualization

The project includes a monitoring and detection mechanism to identify suspicious activities such as port scanning, flooding attempts, abnormal traffic spikes, and unauthorized communication patterns. Captured data is further processed and displayed through a real-time dashboard that provides traffic statistics, protocol distribution, bandwidth usage, and filtering options. The visualization and alerting system helps users quickly understand network conditions and detect potential security threats efficiently.

6. Software/Hardware Required for the Project's Development

Software Requirements

1. Programming Languages

- **Python 3.15+:** Used for packet analysis, traffic processing, and threat detection modules.
- **Go (Golang) 1.25.4+:** Used for high-performance packet capturing, concurrent processing, and backend services.

2. Packet Capturing and Networking Libraries

- **Npcap / libpcap:** Used for low-level packet capturing from network interfaces.
- **gopacket / scapy / pyshark:** Used for packet parsing, protocol decoding, and network traffic analysis.
- **Socket Programming Libraries:** Used for network communication and packet processing operations.

3. Backend Services & Communication

- **REST APIs / WebSockets:** Used for real-time communication between backend services and the dashboard.

4. Development & Deployment Tools

- **Git:** Used for source code management and collaboration.
- **Docker:** Used for containerized deployment and scalability.
- **Postman:** Used for API testing and backend validation.
- **Visual Studio Code:** Used as the primary development environment.

Hardware Requirements

1. Development Machine

- Laptop/PC with at least 8GB RAM: Required for running machine learning models and data analysis. More RAM may be necessary for handling large datasets.
- Processor: Intel i5 or equivalent for standard processing tasks, or higher for more efficient model training.

2. External Storage

- External hard drive (at least 1TB): For backup and data storage.

3. Network Access

- Stable internet connection: Required for real-time data access and API integration.

7. Expected Outcomes

- A fully functional real-time packet sniffing and network monitoring platform capable of capturing and analyzing live network traffic.
- A traffic analysis system providing detailed insights into protocols, IP communication, bandwidth usage, and network activity patterns.
- Operational threat detection functionality capable of identifying suspicious activities such as port scanning, flooding, and abnormal traffic behavior.
- A scalable and modular architecture built using GoLang/Python backend services with real-time dashboard visualization support.
- Successful demonstration and validation of combining Networking, Cybersecurity, and Software Engineering into a practical real-world monitoring solution.

Signature of Student 1–

Signature of Student 2–

Signature of Student 3–

Signature of Student 4–